

Yes. Files can be uploaded through a normal **POST request**. In fact, when you submit a file using an HTML form, the browser is still sending a POST request underneath.

There are two common approaches:

1. HTML Form Upload

HTML



```
<form action="/upload" method="POST" enctype="multipart/form-data">
  <input type="file" name="resume">
  <button type="submit">Upload</button>
</form>
```

The browser automatically creates a `multipart/form-data` POST request.

2. JavaScript Fetch Upload

JavaScript

```
const formData = new FormData()
formData.append("resume", fileInput.files[0])

await fetch("/api/upload", {
  method: "POST",
  body: formData
})
```

No form element is required. The file is sent in the POST request body as `multipart/form-data`.

Flask Backend

Python

Run

```
from flask import request

@app.route("/api/upload", methods=["POST"])
def upload():
    file = request.files["resume"]
    file.save(f"uploads/{file.filename}")
    return {"message": "Uploaded"}
```

For your Placement Portal, resume uploads would typically use:

- Vue `<input type="file">`
- FormData
- `fetch()` with method: "POST"
- Flask `request.files`

Do **not** send files as JSON (`request.json`). Files should be sent using `multipart/form-data` .



”

Since you're building the Placement Portal with Vue + Flask, here's the complete flow from file selection to download.

1. User selects a file in Vue

vue

```
<input
  type="file"
  @change="handleFile"
/>
```

JavaScript

```
import { ref } from "vue"

const selectedFile = ref(null)

function handleFile(event) {
  selectedFile.value = event.target.files[0]
}
```

After selection:

```
JavaScript

console.log(selectedFile.value)
```

Output:

```
JavaScript

File {
  name: "resume.pdf",
  size: 123456,
  type: "application/pdf"
}
```

The browser keeps the actual file contents in memory.

2. Create FormData

Files cannot be serialized with `JSON.stringify()` .

Instead:

```
JavaScript

const formData = new FormData()

formData.append(
  "resume",
  selectedFile.value
)
```

You can also send other fields:

JavaScript

```
formData.append(  
  "student_id",  
  5  
)
```

3. Send to Flask

JavaScript

```
await fetch(  
  `${API_URL}/api/students/resume`,  
  {  
    method: "POST",  
    headers: {  
      Authorization: `Bearer ${token}`  
    },  
    body: formData  
  }  
)
```

Notice:

JavaScript

```
body: formData
```

and **NO**

JavaScript

```
Content-Type
```

The browser automatically creates:

http

```
Content-Type:  
multipart/form-data; boundary=...
```

4. What Flask receives

Request arrives as:

```
http
POST /api/students/resume
multipart/form-data
```

Flask separates:

Files

```
Python Run
request.files
```

Text fields

```
Python Run
request.form
```

Example:

```
Python Run
resume = request.files["resume"]

student_id = request.form["student_id"]
```

5. Save file on disk

```
Python Run
resume.save(
    f"uploads/{resume.filename}"
)
```

Example:

uploads/resume.pdf

6. Better: generate unique names

Otherwise two students upload:

resume.pdf

and overwrite each other.

Use UUID:

Python	Run
<pre>import uuid filename = (str(uuid.uuid4()) + ".pdf") resume.save(f"uploads/{filename}")</pre>	

Result:

```
uploads/
├─ 51c3a.pdf
├─ aa7f9.pdf
├─ 94d11.pdf
```

7. Store path in database

Store only metadata.

Do NOT store the file itself.

Python

Run

```
student.resume_path = (  
    f"uploads/{filename}"  
)
```

Database:

student_id**resume_path**

1

uploads/51c3a.pdf

8. Download later

Student clicks:

HTML



```
<button>  
    View Resume  
</button>
```

Vue:

JavaScript

```
window.open(  
    `${API_URL}/api/students/resume/1`  
)
```

9. Flask sends file

Python

Run

```
from flask import send_file

@app.route(
    "/api/students/resume/<int:id>"
)
def get_resume(id):

    student = Student.query.get(id)

    return send_file(
        student.resume_path
    )
```

Flask reads:

```
uploads/51c3a.pdf
```

and streams it back.

10. Browser receives file

Response:

```
http

Content-Type:
application/pdf
```

Browser:

- Opens PDF
- Or downloads it

depending on headers.

11. Force download

Python

Run

```
return send_file(
    student.resume_path,
    as_attachment=True
)
```

Browser:

Save File

instead of displaying.

12. Protecting files with JWT

You probably don't want:

```
/api/students/resume/1
```

publicly accessible.

Use:

```
Python                                                                    Run

@jwt_required()
def get_resume(id):
```

Now only logged-in users can access.

13. Resume workflow for your project

Upload

```
POST
/api/students/resume
```

Request:

```
multipart/form-data
```

Contains:

```
resume.pdf
```

Backend:

```
uploads/uuid.pdf
```

Store:

```
student.resume_path
```

View

```
GET  
/api/students/resume
```

Backend:

```
PythonRun  
  
return send_file(...)
```

Company views student resume

```
GET  
/api/applications/15/resume
```

Backend:

```
PythonRun  
  
application  
→ student  
→ resume_path
```

Python

Run

```
return send_file(...)
```

Common mistakes

✗ Sending file as JSON

JavaScript

```
body: JSON.stringify(file)
```

Won't work.

✗ Using request.json

Python

Run

```
request.json["resume"]
```

Won't work.

Use:

Python

Run

```
request.files["resume"]
```

✗ Setting Content-Type manually

JavaScript

```
headers: {  
  "Content-Type":  
    "multipart/form-data"  
}
```

Usually breaks uploads.

✗ Storing files in SQLite

Don't do:

```
Python                                     Run

resume_blob = db.Column(...)
```

For this project, storing file paths is much simpler and more practical.

A good Placement Portal design is:

```
Vue
  ↓
FormData
  ↓
POST /api/students/resume
  ↓
Flask
  ↓
request.files["resume"]
  ↓
uploads/uuid.pdf
  ↓
store path in SQLite
```

Later

```
Vue
  ↓
GET /api/students/resume
  ↓
send_file(path)
  ↓
PDF shown/downloaded
```

This is the pattern used by most Flask applications for resumes, profile pictures, documents, and offer letters.

       Sources

Yes, that works, but be careful what `".."` means.

Python

Run

```
os.makedirs("../uploads", exist_ok=True)
```

means:

Create an `uploads` directory one level above the current working directory.

For example, if:

```
placement-portal/  
├─ backend/  
|   ├─ app.py  
|   └─ routes/
```

and your current working directory is:

```
placement-portal/backend
```

then:

Python

Run

```
os.makedirs("../uploads", exist_ok=True)
```

creates:

```
placement-portal/  
├─ uploads/  
└─ backend/
```

To see where it actually resolves to:

Python

Run

```
import os  
  
print(os.path.abspath("../uploads"))
```

Example:

```
/home/prajin/placement-portal/uploads
```

For your project, I would avoid hardcoding "../" and instead do:

Python

Run

```
from pathlib import Path  
  
BASE_DIR = Path(__file__).resolve().parent  
UPLOAD_FOLDER = BASE_DIR.parent / "uploads"  
  
UPLOAD_FOLDER.mkdir(exist_ok=True)
```

This is more reliable because it works regardless of where Flask is launched from.

You can check exactly where your current code will create the folder by printing:

Python

Run

```
print(os.getcwd())  
print(os.path.abspath("../uploads"))
```

before calling `makedirs()`. That will remove any ambiguity.



Pasted text.txt
Document

```
const API_URL = import.meta.env.VITE_API_URL

export async function getStudentProfile() {
  try {
    const access_token =
localStorage.getItem('access_token')
    if (!access_token) {
      return {
        status: 'error',
        message: 'access_token not found',
      }
    }
  }
}
```

Show more ▾

Your frontend code looks mostly correct. One thing I would change:

JavaScript

```
formData.append("resume", profile.resume)
```

If the user doesn't select a new file and `profile.resume` contains:

JavaScript

```
false
```

(from your default profile data) then you'll send:

```
resume=false
```

instead of a file.

Do:

JavaScript

```
if (profile.resume instanceof File) {  
  formData.append("resume", profile.resume)  
}
```

I also noticed a typo:

JavaScript

```
if (data.staus == 'error')
```

should be:

JavaScript

```
if (data.status == 'error')
```

Pasted text

For the backend, since you're using `multipart/form-data`, retrieve fields like this:

Python

Run

```
name = request.form["name"]  
skills = request.form["skills"]
```

and the file:

Python

Run

```
resume = request.files.get("resume")
```

Notice `get()` instead of:

Python

Run

```
request.files["resume"]
```

because the user may update their profile without uploading a new resume.

Then:

Python

Run

```
if resume:
    filename = f"{uuid.uuid4()}.pdf"

    os.makedirs("uploads", exist_ok=True)

    filepath = os.path.join("uploads", filename)

    resume.save(filepath)
```

One more thing for your project design:

When you fetch a profile, don't send the resume file itself:

JavaScript

```
resume: false
```

is fine. Or better:

JavaScript

```
resume_uploaded: true
```

The browser cannot pre-populate:

HTML

```
<input type="file">
```

with an existing file for security reasons.

A common response is:

JSON

```
{
  "name": "Prajin",
  "skills": "Python, Vue",
  "resume_uploaded": true
}
```

Then display:

✓ Resume uploaded

instead of trying to put the file back into the file input.

       Sources

```
from flask import Blueprint, request
from flask_restful import Resource, Api
from flask_jwt_extended import jwt_required,
get_jwt_identity
import models
import uuid
import os
```

```
student_bp = Blueprint("student", __name__, url_prefix="/
api/students")
student_api = Api(student_bp)
```

Show more ▾

Your backend looks fine for the **first upload**, but it has a few problems you'll hit later.

1. files["resume"] can crash

If the user edits their profile without selecting a new resume:

```
Python

resume = files["resume"]
```

Run

raises:

```
BadRequestKeyError: 'resume'
```

Use:

Python

Run

```
resume = request.files.get("resume")
```

Then:

Python

Run

```
filepath = student.resume if student else None
```

```
if resume:
```

```
    filename = f"{uuid.uuid4()}.pdf"
```

```
    os.makedirs("uploads", exist_ok=True)
```

```
    filepath = f"uploads/{filename}"
```

```
    resume.save(filepath)
```

2. Old resumes are never deleted

Suppose a student uploads:

```
uploads/a.pdf
```

Then later uploads:

```
uploads/b.pdf
```

Your database points to `b.pdf`, but `a.pdf` remains forever.

You can delete the old file:

Python

Run

```
if student and student.resume:
    if os.path.exists(student.resume):
        os.remove(student.resume)
```

before saving the new one.

3. Relative path issue

This:

Python

Run

```
filepath = f"uploads/{filename}"
```

depends on where Flask is launched from.

Safer:

Python

Run

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))

UPLOAD_FOLDER = os.path.join(BASE_DIR, "..", "uploads")

os.makedirs(UPLOAD_FOLDER, exist_ok=True)

filepath = os.path.join(
    UPLOAD_FOLDER,
    filename
)
```

4. You should validate the file type

Currently someone can upload:

```
virus.exe
```

and you save it as:

```
uuid.pdf
```

Check:

Python

Run

```
if resume:
    if resume.mimetype != "application/pdf":
        return {
            "status": "error",
            "message": "Resume must be PDF"
        }, 400
```

5. GET response

Right now:

Python

Run

```
"resume": student.resume
```

returns:

JSON

```
{
  "resume": "uploads/abc.pdf"
}
```

I would return:

Python

Run

```
"resume_uploaded": bool(student.resume)
```

because the frontend cannot populate a file input from a path anyway.

6. CGPA type

You're storing:

Python

Run

```
cgpa = form["cgpa"]
```

which is a string.

If your model column is numeric:

Python

Run

```
cgpa = float(form["cgpa"])
```

The biggest improvement I'd make immediately is:

Python

Run

```
resume = request.files.get("resume")
```

because otherwise profile updates will fail whenever the user doesn't upload a new resume. That's the first bug you'll encounter in production.

